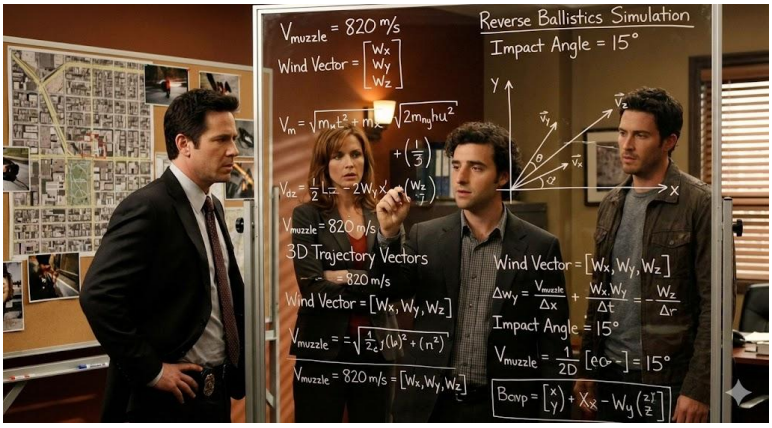
Ballisztikai elemzés – Muzzle velocity

Vektorok

A Sztori: "A Golyó Emlékezete"



Helyszín: FBI Iroda, tárgyaló. A falon a város térképe. Szereplők: Charlie Eppes, Don Eppes (FBI ügynök), Megan Reeves (FBI viselkedéselemző) és Colby Granger (FBI ügynök).

Don: "Charlie, ez tű a szénakazalban. Van egy golyónk a falban, egy halottunk, és nulla szemtanúnk. A lövés jöhetett bármelyik tetőről, bármelyik ablakból két kilométeres körzetben. A helyszínelők szerint a becsapódás szöge 15 fok volt, de a szél miatt ez bármi lehetett."

Charlie: "Nem, Don. A fizika szimmetrikus. Ha ismerjük a végpontot, a szelet és a gravitációt, akkor az időt visszafelé pörgetve a golyó elvezet minket a fegyver csövéig. A golyó 'emlékszik' arra, honnan jött, csak ki kell olvasnunk a vektorokból. Nem keresgélünk. Visszaszámolunk."

Colby: "Charlie, átnéztük az összes tetőt. Üresek. Nincs lőállás, nincs nyom. Lehet, hogy tévedtünk a távolsággal?"

Charlie: "Nem tévedtünk, csak rossz változót rögzítettünk. A távolság nem konstans, Colby. Az energia a konstans. A ballisztikai labor azonosította a fegyvert egy Remington 700-asnak. Ennek a puskának a torkolati sebessége (muzzle velocity) fix. A golyó addig repült, amíg a légellenállás le nem lassította a becsapódási sebességre. Ha visszafelé szimuláljuk az utat, és figyeljük, mikor éri el a golyó a torkolati sebességet... ott, abban a pillanatban, és abban a térbeli pontban dördült el a lövés."

Megan: "Charlie, a meteorológiai jelentés szerint viharos szél volt aznap. De nem szembeszél. A szél oldalról-fentről csapott le, 'bukószél' volt az épületek között. Ez nem változtat mindenen?"

Charlie: "Dehogynem, Megan. A szél nem csak lassít vagy gyorsít. A szél egy vektor. Van ereje (sebessége) és van iránya (szöge). Ha a szél fentről fúj, az a gravitációval együtt dolgozik, és lenyomja a golyót. Ha mi visszafelé megyünk az időben, ezt a vektort kell kivonnunk a golyó mozgásából. A golyó 'küzdött' a széllel. Nekünk vissza kell adnunk neki azt az energiát, amit a szél elvett tőle."

Feladat

Minden fegyvernek van egy ujjlenyomata: a torkolati sebesség. Ha tudjuk, hogy egy M24-es mesterlövész puskát használtak, akkor tudjuk, hogy a golyó 853 m/s sebességgel hagyta el a csövet. Csak addig kell pörgetnünk vissza az időt, amíg a golyó sebessége el nem éri ezt a számot. Ahol ez megtörténik, ott húzták meg a ravaszt.

A te feladatod megírni Charlie algoritmusát C# nyelven. Adott a becsapódás helye, a golyó sebessége és a környezeti adatok. Szimuláld a lövedék útját visszafelé, amíg meg nem találjuk a lövész pozícióját rajzold ki az útvonalat a konzolra!

Bemeneti Adatok (például):

- **Becsapódás**:
  - **Helye (Áldozat), kezdő pozíció:** X = 600m, Y = 1.5 m, Z=300 m
  - **Sebesség:** 819 m/s
  - **Bólintás (Pitch/Elevation):** -2.15^{o} (Fentről lefelé érkezett)
  - **Irány (Yaw/Azimuth):** 45° (Északkeleti irányból csapódott be)
- **Környezeti hatások:**
  - **Szélsebesség:** 8 m/s
  - **Szélsebesség szöge:** 350°
  - **Gravitáció:** -9.81 m/s^2
  - **Szélellenállás:** 0.1
- **Fegyver Torkolati Sebessége:** 820 m/s

```
LÖVÉSZ POZÍCIÓJA:
  X = -1126,1 m
  Z = -1421,2 m
  Y = 62,7 m (Magasság)
```

1. Matematikai Alapok – Vektor3D

1.1. Feladat: Készíts egy Vektor3D struktúrát: legyen X, Y és Z adattagja.

1.2. Feladat: Írj egy függvényt, ami a térbeli vektort két szögből (Pitch, Yaw) állítja elő a sebességvektort (velocity):

1.2.1. V_y=BecsapódásiSebesség * sin(Pitch) *(A magasságnál csak a bólintás számít).*

1.2.2. A vízszintes sebesség komponens: $V_{sík}$: BecsapódásiSebesség * cos(Pitch).

1.2.3. V_x=$V_{sík}$ * sin(Yaw)

1.2.4. V_z=$V_{sík}$ * cos(Yaw)

1.3. Feladat: Írj egy függvényt, ami meghatározza a vektor hosszát: $\sqrt{x^2 + y^2 + z^2}$

1.4. Feladat: Írj egy függvényt, ami összead két vektort!

1.5. Feladat: Írj egy függvényt, ami kivon két vektort!

1.6. Feladat: Írj egy függvényt, ami megszoroz egy vektort egy skalárral!

*Megjegyzés: Figyelj a radián átváltásra: radián=fok * π / 180! Érdemes a vektor meghatározást is külön függvényekben megvalósítani!*

2. A Fizikai Motor (Térbeli Visszapörgetés)

2.1. Feladat: Haladj visszafelé (dt = 0.01s), amíg a becsapódási sebesség el nem éri a torkolati sebességet (a sebességvektor hossza, az aktuális sebesség). Az útvonalat tárold egy listában! **Erők kezelése visszafelé:**

2.1.1. **Pozíció:** pos = pos - velocity * dt; (Minden tengelyen lépünk).

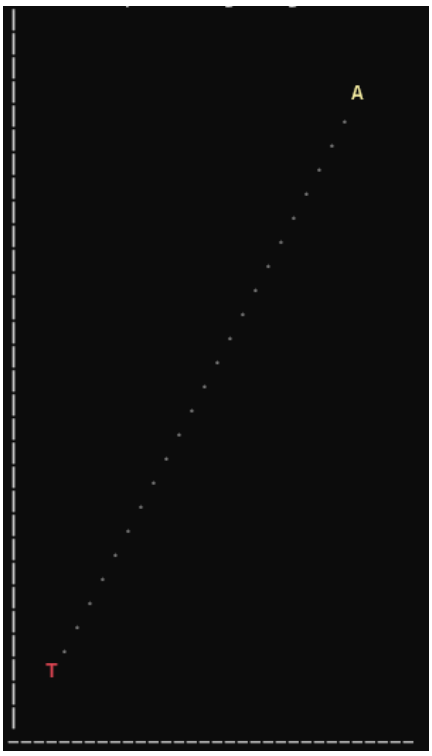
2.1.2. **Gravitáció:** Csak az **Y** tengelyen hat: velocity = velocity - (gravitacioVektor * dt);

2.1.3. **Szél:** A szél vízszintesen fúj (X és Z tengelyen). A fenti képlethez hasonlóan bontsd fel a szelet is X és Z komponensre a szögéből! Majd vond ki a szélvektort a golyó sebességéből (megfelelő együtthatóval, pl. légellenállás), hogy "visszagyorsítsd" a golyót vízszintesen. velocity = velocity + (velocity -szel) * dt * legellenallas;

```
--- BALLISZTIKAI ELEMZÉS (REVERSE MODE) ---

1. (600,00,1,50,300,00)
A golyóból kivontuk a sebességvektort: (595,91;1,81;295,91)
A sebeségvektort módosítjuk a gravitációval: (409,50;-30,63;409,50)
A sebeségvektort módosítjuk a környezeti adatokkal: (409,91;-30,66;409,90)
2. (595,91,1,81,295,91)
A golyóból kivontuk a sebességvektort: (591,81;2,11;291,81)
A sebeségvektort módosítjuk a gravitációval: (409,91;-30,56;409,90)
A sebeségvektort módosítjuk a környezeti adatokkal: (410,32;-30,59;410,30)
3. (591,81,2,11,291,81)
A golyóból kivontuk a sebességvektort: (587,70;2,42;287,70)
A sebeségvektort módosítjuk a gravitációval: (410,32;-30,49;410,30)
A sebeségvektort módosítjuk a környezeti adatokkal: (410,73;-30,52;410,71)
4. (587,70,2,42,287,70)
A golyóból kivontuk a sebességvektort: (583,60;2,72;283,60)
A sebeségvektort módosítjuk a gravitációval: (410,73;-30,42;410,71)
A sebeségvektort módosítjuk a környezeti adatokkal: (411,14;-30,46;411,11)
5. (583,60,2,72,283,60)
A golyóból kivontuk a sebességvektort: (579,48;3,03;279,48)
A sebeségvektort módosítjuk a gravitációval: (411,14;-30,36;411,11)
A sebeségvektort módosítjuk a környezeti adatokkal: (411,55;-30,39;411,51)
6. (579,48,3,03,279,48)
A golyóból kivontuk a sebességvektort: (575,37;3,33;275,37)
A sebeségvektort módosítjuk a gravitációval: (411,55;-30,29;411,51)
A sebeségvektort módosítjuk a környezeti adatokkal: (411,96;-30,32;411,92)
```

3. A Vizualizáció (Felülnézeti Térkép)



Mivel a magasságot nem ábrázoljuk, a konzolra az X és Z koordinátákat kell kirajzolnod. Ez olyan, mintha egy drónról néznénk a várost.

3.1. Feladat: Rajzold ki normalizálva (skálázva) a térképet úgy, hogy 50 x 50-es karakteres képbe beleférjen. (A vízszintes tengely a konzolon: X koordináta, függőleges tengely a konzolon: Z koordináta). **Jelölések:**

- A: Áldozat (Startpont: X=500, Z=300)
- T: Tettes (Ahol a szimuláció megállt)
- .: A röppálya vetülete (az útvonal a földre vetítve).

Clean Code Elvárások: „A Tiszta Kód Szabályai”

A megoldás értékelésekor nem csak a működésre figyelj oda, hanem a kód minőségére is. Tartsd be az alábbi szabályokat:

1. Névadás (Naming Conventions)

Beszédes nevek: A változók és függvények nevei árulják el, hogy mire valók!

Rossz: a, b, t, x (kivéve koordinátáknál), func1

Jó: matrixMeret, buntenyekSzama, tavolsagSzamitas, koronBeluliEsetek

Ige-Főnév a függvényeknél: A függvény csinál valamit, ezért a nevében legyen ige.

Példa: MatrixFeltoltes, EredmenyKiiras, TavolsagSzamitas.

Nyelv: Válassz egy nyelvet (magyar vagy angol), és használd konzekvensen végig! Ne keverd (pl. calculateTavolsag).

2. Függvények felépítése

Egy függvény = Egy feladat (SRP): Egy függvény csak egyetlen dolgot csináljon, de azt tegye tökéletesen. Ha a függvényed neve „...És...” szót tartalmaz (pl. FeltoltesEsKiiras), akkor azt szét kell szedni két külön függvényre!

Rövid függvények: Törekedj arra, hogy egy függvény (ha lehet) ráférjen a képernyőre görgetés nélkül (kb. 10-20 sor).

Logikus sorrend: A kód legyen fentről lefelé olvasható. A segédfüggvények (amiket meghívsz) legyenek definiálva a hívás helye felett (vagy logikusan csoportosítva), hogy a kódot olvasva lássuk az építőkockákat, mielőtt felhasználjuk őket a Main-ben.

3. Adatkezelés és Paraméterek

NINCS globális változó: Tilos a függvényeken kívül, az osztály szintjén változót deklarálni (kivéve a konstansokat). Minden adatot paraméterként kell átadni a függvényeknek! A függvény csak azt láthatja, amit megkapott.

4. Bonyolultság és Logika

Mélység (Nesting): A ciklusok és elágazások egymásba ágyazása ne haladja meg a 3 szintet!

Kerüld: for → for → if → if (ez már 4 szint).

Megoldás: Használj korai visszatérést (return) vagy szervezd ki a belső részt egy új függvénybe.

Bonyolult feltételek kiszervezése: Ha egy if feltétel nem fér ki kényelmesen egy sorba, vagy zárójelek erdejét látod, szervezd ki egy beszélő nevű bool visszatérésű függvénybe.

Rossz: if (x >= 10 && x+y < M && y-1 >= 0 && y+2 < N && matrix[x,y] == 10 && x!=y)

Jó: if (ErvenyesBunteny(x, y, matrix, n, m))

5. Kommentelés

Ne magyarázd a nyilvánvalót: Ne írj olyat, hogy i++; // növeljük i-t.

A "Miért"-et kommenteld, ne a "Hogyan"-t: A kód azt mutatja, hogyan csinálod, a komment azt, hogy miért úgy döntöttél (pl. matematikai képlet forrása). A beszédes függvénynevek feleslegessé teszik a kommentek 90%-át.